

Data-Oriented Design vs. Object-Oriented Structures in High-Performance Systems Programming

System Programming Course



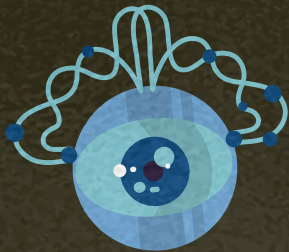
*Spyridon Arvanitis, Dimitrios
Xiromeritis & Alexios Politis*



INTRODUCTION & MICROARCHITECTURE

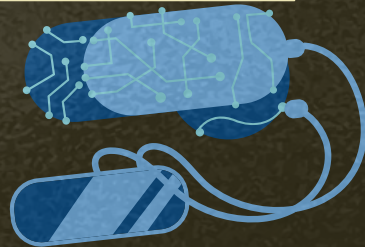
Navigating the "Memory Wall" and CPU Cache Hierarchies





The "Memory Wall" Problem

CPU execution throughput and clock frequencies have advanced much faster than system memory (DRAM) access latencies. Modern microprocessors rely heavily on complex, tiered hierarchical caching (L1, L2, and shared L3 SRAM) to combat this latency mismatch. Whenever a "cache miss" occurs, the core pipeline halts execution for hundreds of clock cycles



Microarchitectural Mechanics

- ❑ The CPU memory controller reads data in discrete, atomic segments known as cache lines, standardized at 64 bytes
- ❑ Hardware "spatial prefetchers" proactively analyze and load sequential memory addresses into the cache ahead of schedule

- ❑ Traditional Object-Oriented Programming (OOP) frequently degrades this efficiency by bundling data fields and behaviors directly inside conceptual entities



MEMORY LAYOUT PARADIGMS (AoS VS. SoA)

Restructuring Data for Maximum Cache Efficiency



Array of Structures (AoS) - The OOP Paradigm

64-Byte Footprint

The total size of each AoS particle entity was expanded to 64 bytes



Dead Weight Payload

During a simple position update, only 16 bytes are used. The remaining 48 bytes are read into the L1 cache as unnecessary dead weight



I-Cache Isolation

Dynamic virtual dispatch and internal function pointers were intentionally omitted to strictly isolate the performance bottleneck to the Data Cache



Structure of Arrays (SoA)

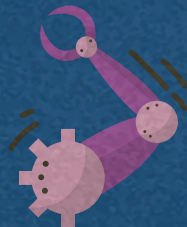
The conceptual entity is disassembled completely. Components are extracted into individual, homogenous parallel scalar tables



EXPLICIT SIMD VECTORIZATION & OPTIMIZATION



Leveraging AVX Pipelines and Branchless Logic



Vectorization Architecture

AVX/AVX2 Intrinsics

The SoA pipeline was augmented with explicit loop vectorization to process multiple particles simultaneously



Strict Alignment

Required strict 64-byte base address alignment to prevent costly, overlapping split memory bus requests

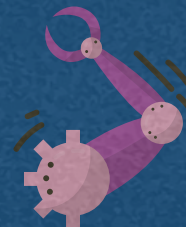
Branchless Logic

The physics pipeline was made entirely branchless using bitwise masks, eliminating Branch Prediction Unit (BPU) mispredictions

EXPERIMENTAL METHODOLOGY & SETUP



Architectural Isolation and Benchmarking Parameters



Testing Environment



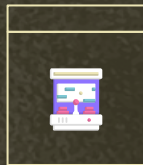
Hardware

Executed natively on an Intel Core i7-13620H micro-architecture (2.4 GHz)



Workloads

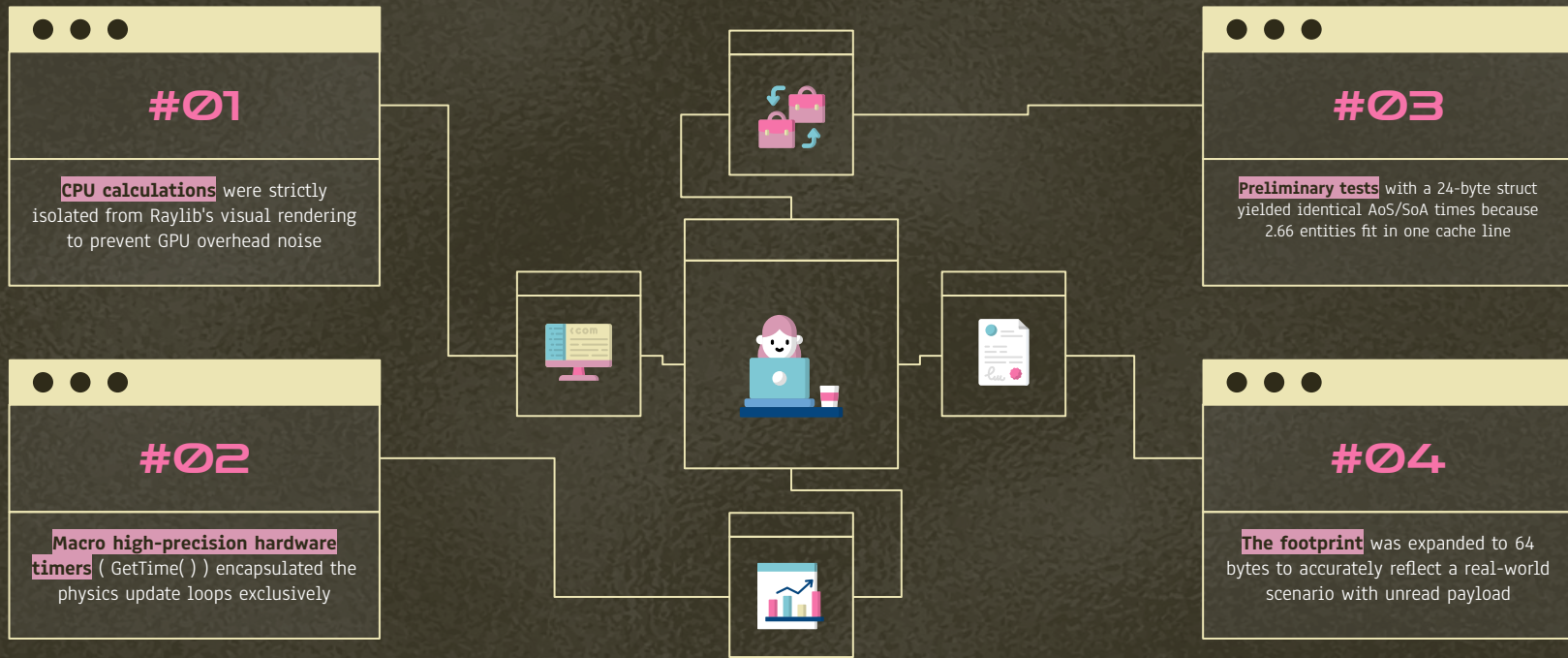
Scaled massive datasets up to 2,000,000 active execution nodes



Cache Limits

Constrained to 48 KB of L1 Data Cache per core (exactly 768 discrete 64-byte cache lines)

Profiling Isolation & The Small-Struct Anomaly



EMPIRICAL ANALYSIS & PERFORMANCE METRIC

Tracing Hardware Cache Misses and Execution Latency



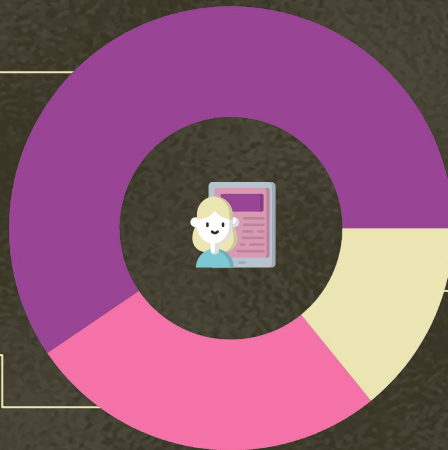
Hardware Memory Trace Analysis (2M Nodes)

**AoS (ΘΘP) - 123.5M
Misses**

The bloated AoS structure forced the CPU to generate over 5 billion data memory references

**SoA (Scalar) - 54.7M
Misses**

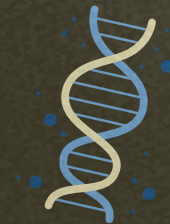
Data references were cut by more than half, drastically dropping L1 misses



**SoA (SIMD) - 29.4M
Misses**

Vectorization compressed the memory footprint even further, reducing misses to an absolute minimum

Instruction Pipeline Efficiency

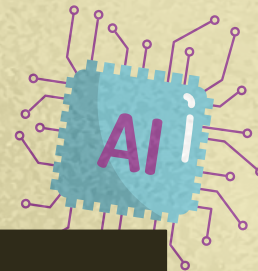


**13.5 Billion
Reads (AoS)**

**6.2 Billion
Reads (SoA)**

**3.0 Billion
Reads (SIMD)**

Because DOD separates data into uniform streams, the compiler generates significantly cleaner assembly code



1.74 ms

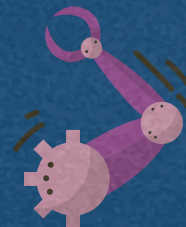
By combining SoA data density with AVX SIMD vectorization, execution time dropped from 9.09 ms to 1.74 ms per frame—an enormous 5.2x speedup



CONCLUSIONS & TAKEAWAYS



The Performance Impact of Hardware-Conscious Engineering



Final Takeaways

Massive Throughput

Reorganizing data structures to respect the physical mechanics of the L1 cache achieved a 520% throughput increase

Bypassing the Wall

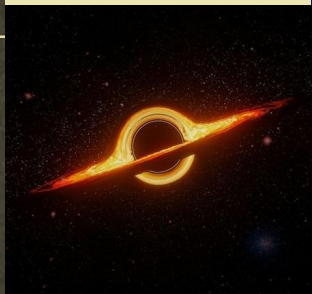
Hardware-conscious software engineering allows **high-performance systems** to successfully bypass the "Memory Wall"

Data Organization

Organizing data or hardware consumption is demonstrated to be just as critical to application performance as algorithmic complexity

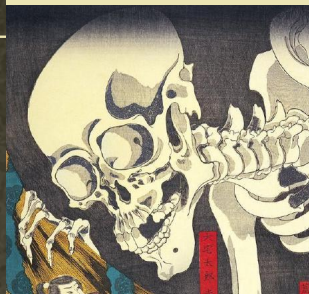
Our team

Arvanitis



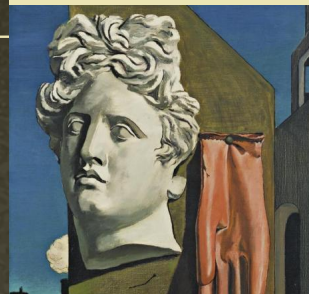
Spyridon

Xiromeritis



Dimitrios

Politis



Alexios



THANKS!

CREDITS: This presentation template was created by
Slidesgo, including icons by **Flaticon**, and
infographics & images by **Freepik**